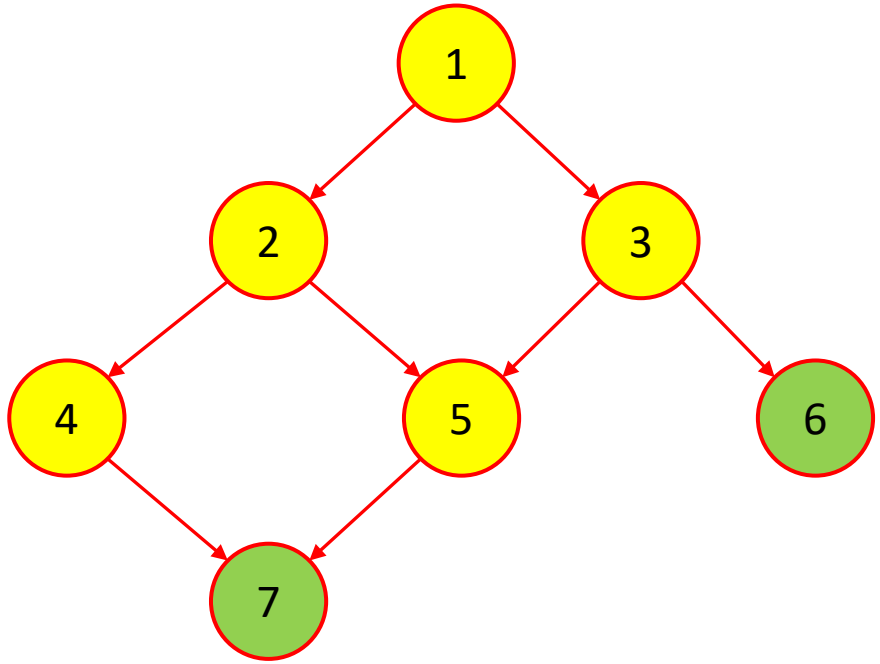
Question

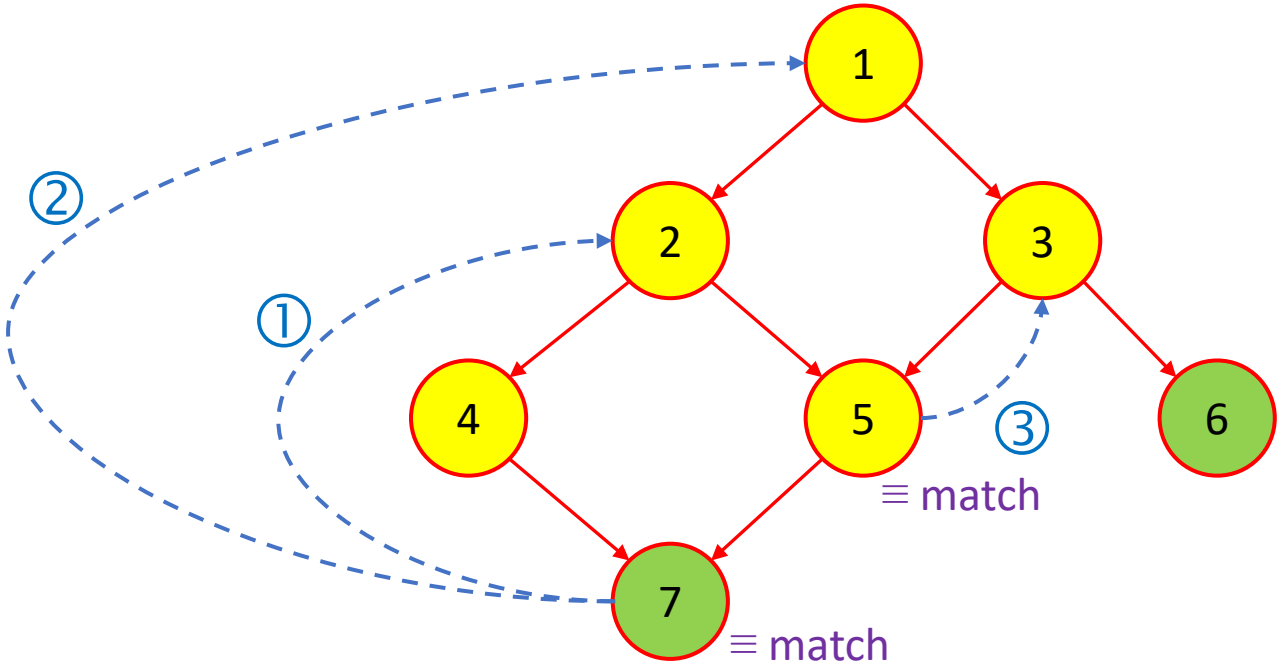
Question> Given the following state transition diagram, there are three backtracking arrows, when doing a depth-first-search (DFS) state space exploration. Fill in the blank start and end states in the table.



Backtracking Arrows	Start State	End State
First arrow		
Second arrow		
Third arrow		

Answer

Answer> States are visited in the order shown. When there is a match or termination, model checker backtracks to nearest transition pending exploration. In this way, each unique state is visited only once in the exploration.



Backtracking Arrows	Start State	End State
First arrow	7	2
Second arrow	7	1
Third arrow	5	3

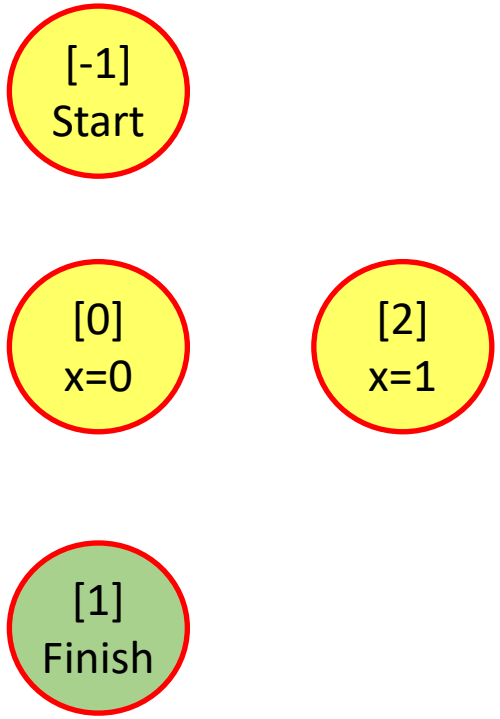
With matching & backtracking: 1 → 2 → 4 → 7 → 5 → 3 → 6

Without matching & backtracking: 1 → 2 → 4 → 7 → 1 → 2 → 5 → 7 → 1 → 3 → 5 → 7 → 1 → 3 → 6

Question

Question> Given the following code, JPF stores four states: -1, 0, 1, 2. Draw the transition edges yourself (including the backtracking edges) and then fill in the backtracking arrow table.

```
x = 0;
while(true) {
  x = rand.nextInt(2);
  if(x == 0) break;
}
```

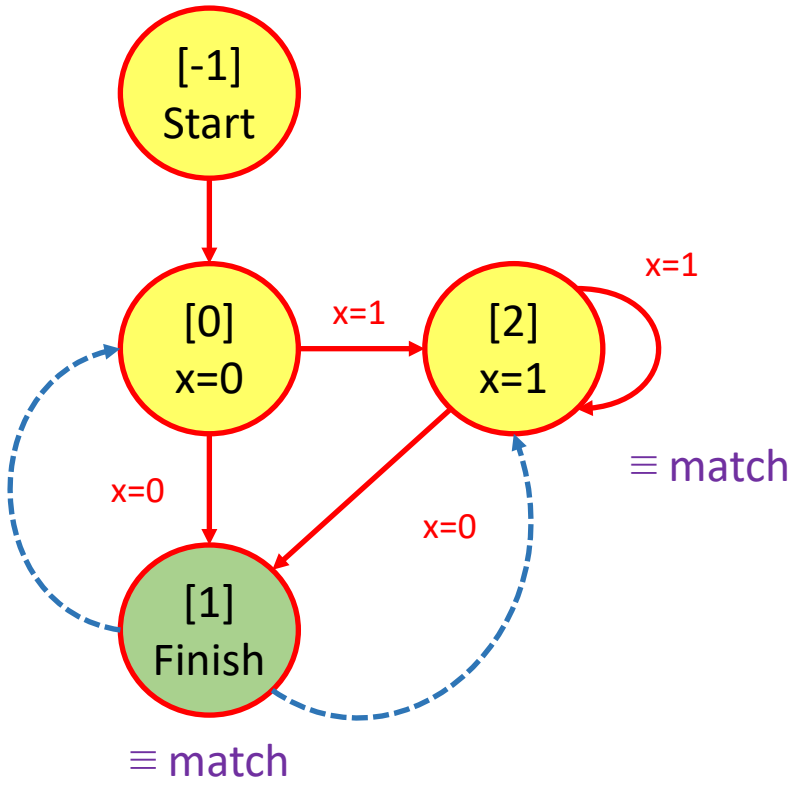


Backtracking Arrows	Start State	End State
First arrow		
Second arrow		
Third arrow		

Answer

Answer> JPF stores a state wherever it chooses x, so x=0 leads straight to the end. Without matching & backtracking, [2] would be revisited forever.

```
x = 0;
while(true) {
  x = rand.nextInt(2);
  if(x == 0) break;
}
```



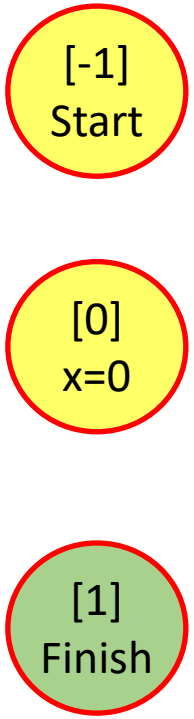
Backtracking Arrows	Start State	End State
First arrow	1	0
Second arrow	1	2
Third arrow	N/A	N/A

Order of visitation: -1 → 0 → 1 → 2

Question

Question> Given the following code, JPF stores three states: -1, 0, 1. Draw the transition edges yourself (including the backtracking edges) and then fill in the backtracking arrow table.

```
x = 0;
while(true) {
  x = rand.nextInt(2);
  if(x == 1) break;
}
```

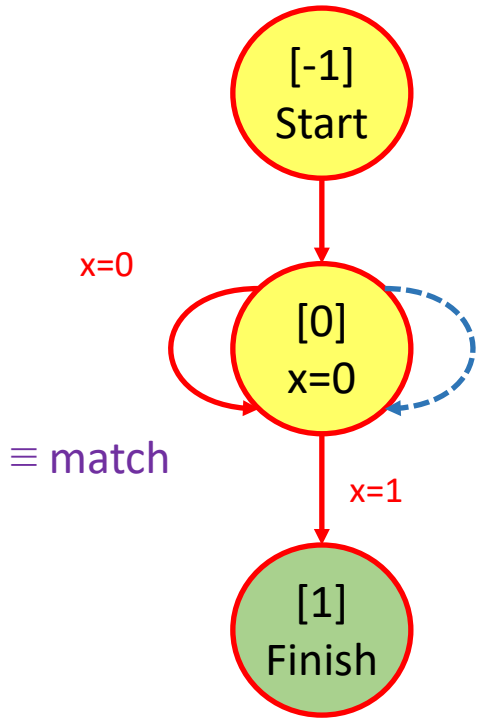


Backtracking Arrows	Start State	End State
First arrow		
Second arrow		
Third arrow		

Answer

Answer> Choosing x=0 sets x to 0 again, so JPF is right back in state [0]. Without matching & backtracking, it would loop there forever.

```
x = 0;
while(true) {
  x = rand.nextInt(2);
  if(x == 1) break;
}
```



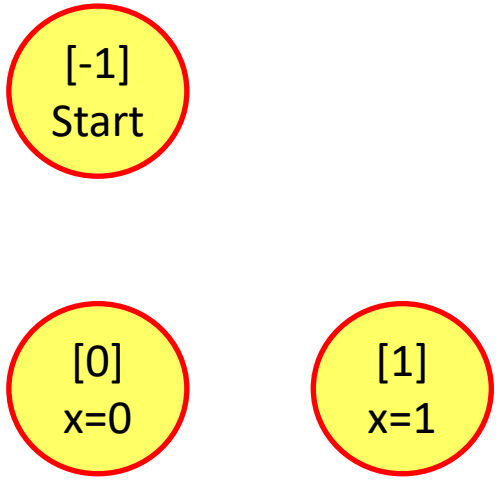
Backtracking Arrows	Start State	End State
First arrow	0	0
Second arrow	N/A	N/A
Third arrow	N/A	N/A

Order of visitation: -1 → 0 → 1

Question

Question> Given the following code, JPF stores three states: -1, 0, 1. Draw the transition edges yourself (including the backtracking edges) and then fill in the backtracking arrow table.

```
x = 0;
while(true) {
  x = rand.nextInt(2);
}
```

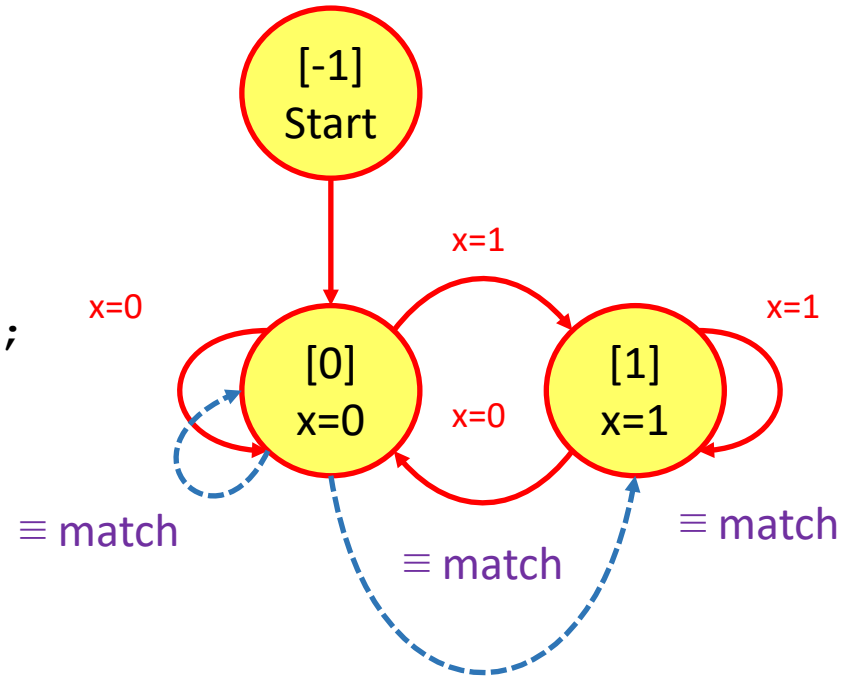


Backtracking Arrows	Start State	End State
First arrow		
Second arrow		
Third arrow		

Answer

Answer> Again, without matching & backtracking, the state space would expand infinitely due to repeated visits to [0] and [1].

```
x = 0;
while(true) {
  x = rand.nextInt(2);
}
```



Backtracking Arrows	Start State	End State
First arrow	0	0
Second arrow	0	1
Third arrow	N/A	N/A

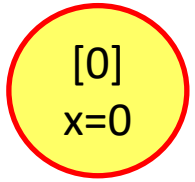
Order of visitation: -1 → 0 → 1

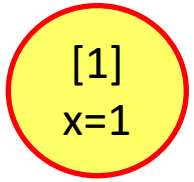
Question

Question> Given the following code, JPF stores three states: -1, 0, 1, because it ends a transition after every 50,000 instructions. Draw the transition edges yourself (including the backtracking edges) and then fill in the backtracking arrow table.

```
x = 0;
while(true) {
  x = (x + 1) % 2;
}
```





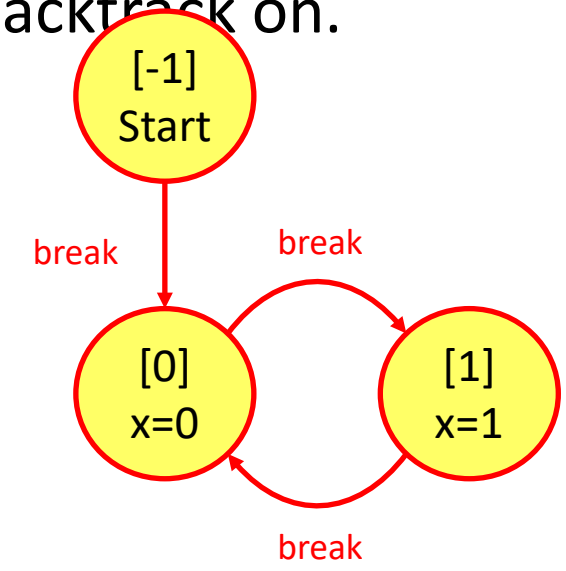


Backtracking Arrows	Start State	End State
First arrow		
Second arrow		
Third arrow		

Answer

Answer> With no choices, JPF ends a transition only every 50,000 instructions (a break), so each arrow covers thousands of iterations. There are no pending choices to backtrack on.

```
x = 0;
while(true) {
  x = (x + 1) % 2;
}
```



≡ match

Backtracking Arrows	Start State	End State
First arrow	N/A	N/A
Second arrow	N/A	N/A
Third arrow	N/A	N/A

Order of visitation: -1 → 0 → 1